

Système d'exploitation

Système d'exploitation - Utilisation

Mickael Rouvier

CERI – Avignon Université
`mickael.rouvier@univ-avignon.fr`

March 2, 2026

Section 1

Gestion des droits des fichiers

Droits d'accès aux fichiers

- **Problématique** : Comment définir efficacement les droits d'accès aux fichiers ?
- **Solution idéal** : Définir un ensemble complet de droits d'accès pour chaque combinaison possible d'utilisateurs et de modes d'utilisation.
 - Façons d'utiliser un fichier (lecture, écriture, exécution...)
 - Classes d'utilisateurs (propriétaire, groupe, autres)
 - **Impossible** : Nombre de combinaisons trop grand à gérer manuellement
- **Approche Réaliste** :
 - 3 modes d'utilisation (droits) :
 - Lecture (**r**)
 - Écriture (**w**)
 - Exécution (**x**) ou traversée d'un répertoire
 - 3 classes d'utilisateurs :
 - Propriétaire du fichier
 - Groupe auquel appartient le propriétaire
 - Tous les autres utilisateurs

Définition : Utilisateur et Groupe

- **Utilisateur** : Toute entité (personne physique ou programme) devant interagir avec un système UNIX est authentifiée sur cet ordinateur par un utilisateur (user).
- **Groupe** : Un utilisateur UNIX appartient à un ou plusieurs groupes. Les groupes permettent de regrouper des utilisateurs afin de leur attribuer des droits communs.

```
invite/> ls -l
-rw-r--r--@ 1 rouvier staff 3613 12 nov 08:02 Archive.zip
-rw-r--r--@ 1 rouvier staff 3613 12 nov 08:02 Archive_cnn.zip
drwxr-xr-x@ 5 rouvier staff 3613 12 nov 08:02 Chess-Engine
-rw-r--r--@ 1 rouvier staff 3613 12 nov 08:02 test.dic
-rw-r--r--@ 1 rouvier staff 3613 12 nov 08:02 toto.py
```

Affichage des droits d'accès

- **Format des Droits** : Affichés sur 10 caractères
 - 1er caractère : Type de fichier
 - - : Fichier ordinaire
 - d : Répertoire
 - l : Lien symbolique
 - 2 à 10 : Droits d'accès
 - Utilisateurs : Lecture (**r**), Écriture (**w**), Exécution (**x**)
 - Classes : Propriétaire (**u**), Groupe (**g**), Autres (**o**)
- **Exemple** :

```
invite/> ls -l
-rw-r--r-- 1 rouvier staff 3613 12 nov 08:02 Archive.zip
drwxr-xr-x 5 rouvier staff 3613 12 nov 08:02 Chess-Engine
```

Exemples droits de fichiers

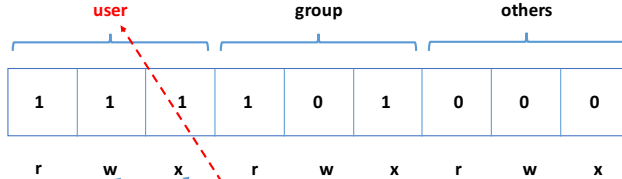
user			group			others		
1	1	1	1	0	1	0	0	0
r	w	x	r	w	x	r	w	x

Explications : Fichier que

- le propriétaire peut lire, écrire et exécuter
- ceux du groupe peuvent lire et exécuter
- les autres ne peuvent rien faire dessus

Affichage : -rwxr-x---

Exemples droits de fichiers



Explications : Fichier que

- le **propriétaire** peut **lire**, **écrire** et **exécuter**
- ceux du groupe peuvent lire et exécuter
- les autres ne peuvent rien faire dessus

Affichage : -rwxr-x---

Exemples droits de fichiers

user			group			others		
1	1	1	1	0	1	0	0	0
r	w	x	r	w	x	r	w	x

Explications : Fichier que

- le propriétaire peut lire, écrire et exécuter
- ceux du **groupe** peuvent **lire** et **exécuter**
- les autres ne peuvent rien faire dessus

Affichage : -rwxr-x---

Exemples droits de fichiers

user			group			others		
1	1	1	1	0	1	0	0	0
r	w	x	r	w	x	r	w	x

Explications : Fichier que

- le propriétaire peut lire, écrire et exécuter
- ceux du groupe peuvent lire et exécuter
- les **autres** ne peuvent rien faire dessus

Affichage : -rwxr-x---

Changement des droits d'un fichier

- **Commande** : `chmod` (**ch**ange **mo**de)
- **Méthodes d'Utilisation** :
 - **Symbolique** : `chmod [ugoa] [+ -=] [rwx] fichier`
 - **Octale** : `chmod XXX fichier`

Changement des droits d'un fichier – symbolique

- **Syntaxe** : `chmod [ugoa] [+ -=] [rwx] fichier`
- **Classes** :
 - **a** : Tous (user, group, others) (défaut)
 - **u** : Propriétaire (user)
 - **g** : Groupe
 - **o** : Autres (others)
- **Opérations** :
 - **+** : Ajouter des droits
 - **-** : Retirer des droits
 - **=** : Affecter des droits spécifiques
- **Droits** :
 - **r** : Lecture (read)
 - **w** : Écriture (write)
 - **x** : Exécution (execute) ou traversée d'un répertoire

Changement des droits d'un fichier - symbolique

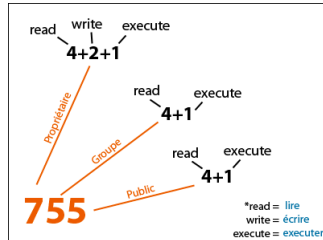
Exemple :

```
invite/> ls -l toto.py
-rw-r--r-- 1 rouvier staff 486 29 jan 11:57 toto.py
invite/> chmod a+rx toto.py
invite/> ls -l toto.py
-rwxrwxrwx 1 rouvier staff 486 29 jan 11:57 toto.py
```

- `chmod a+rx toto.py` : Ajoute les droits de lecture, écriture et exécution pour tous (user, group, others).

Changement des droits d'un fichier - octal

- **Syntaxe** : `chmod XXX fichier`
- **Mode Octal** :
 - Chaque chiffre représente les droits pour user, group, et others respectivement.



Changement des droits d'un fichier - octal

- **Syntaxe** : `chmod XXX fichier`
- **Mode Octal** :
 - Chaque chiffre représente les droits pour user, group, et others respectivement.
 - **Exemples de Codes Octaux** :
 - **644** : Lecture/Écriture pour le propriétaire, Lecture pour le groupe et les autres.
 - **766** : Lecture/Écriture/Exécution pour le propriétaire, Lecture/Écriture pour le groupe et les autres.
 - **750** : Lecture/Écriture/Exécution pour le propriétaire, Lecture/Exécution pour le groupe, Aucun droit pour les autres.
- **Exemple** :

```
invite/> ls -l toto.py
-rw-r--r-- 1 rouvier staff 486 29 jan 11:57 toto.py
invite/> chmod 777 toto.py
invite/> ls -l toto.py
-rwxrwxrwx 1 rouvier staff 486 29 jan 11:57 toto.py
```

- `chmod 777 toto.py` : Donne tous les droits (lecture, écriture, exécution) au propriétaire, au groupe et aux autres.

Section 2

Manipulation des fichiers

Commandes de base sur les fichiers

- **basename** : Récupère le nom du fichier sans le chemin ou le suffixe
- **file** : Détermine le type d'un fichier
- **cat** : Affiche le contenu d'un ou plusieurs fichiers
- **head** : Affiche les premières lignes d'un fichier
- **tail** : Affiche les dernières lignes d'un fichier
- **touch** : Modifie les timestamps d'un fichier ou en crée un vide
- **more** : Affiche le contenu d'un fichier page par page

basename

Description : La commande `basename` extrait le nom du fichier sans le chemin complet ou le suffixe spécifié.

Syntaxe :

```
basename [chemin_fichier] [suffixe]
```

Exemples :

```
invite/> basename /etc/fichier.sh  
fichier.sh
```

```
invite/> basename /etc/fichier.sh .sh  
fichier
```

Description : La commande `file` détermine le type d'un fichier en analysant son contenu.

Syntaxe :

```
file [nom_fichier]
```

Exemples :

```
invite/> file fichier.gz  
fichier.gz: gzip compressed data, from Unix, max compression
```

```
invite/> file 64x64x32.png  
64x64x32.png: PNG image, 64 x 64, 8-bit/color RGBA, non-interlaced
```

Description : La commande `cat` affiche le contenu d'un ou plusieurs fichiers directement dans le terminal.

Syntaxe :

```
cat [fichier1] [fichier2] ...
```

Exemples :

```
invite/> cat fichier.txt  
Bla, bla, bla !!!!  
XXXXX
```

```
invite/> cat fichier1.txt fichier2.txt  
Bla, bla, bla !!!!  
XXXXX  
Bli, bli, bli !!!!  
YYYY
```

head

Description : La commande `head` affiche les premières lignes d'un fichier.

Syntaxe :

```
head [options] [fichier]
```

Exemples :

```
invite/> head fichier.txt  
bla1, bla1, bla1 !!!!  
bla2, bla2, bla2 !!!!  
bla3, bla3, bla3 !!!!  
bla4, bla4, bla4 !!!!
```

```
invite/> head -n 1 fichier2.txt  
bla1, bla1, bla1 !!!!
```

Description : La commande `tail` affiche les dernières lignes d'un fichier.

Syntaxe :

```
tail [options] [fichier]
```

Exemples :

```
invite/> tail fichier.txt  
bla1, bla1, bla1 !!!!  
bla2, bla2, bla2 !!!!  
bla3, bla3, bla3 !!!!  
bla4, bla4, bla4 !!!!
```

```
invite/> tail -n 1 fichier2.txt  
bla4, bla4, bla4 !!!!
```

touch

Description : La commande **touch** permet de modifier les timestamps de dernier accès et de dernière modification d'un fichier. Si le fichier n'existe pas, **touch** le crée vide.

Syntaxe :

```
touch [options] [fichier]
```

Exemples :

```
invite/> touch fichier.txt
```

```
invite/> ls -lrth fichier.txt
-rw-r--r-- 1 rouvier staff 486 21 jan 11:57 toto.py
invite/> touch fichier.txt
-rw-r--r-- 1 rouvier staff 486 28 jan 13:00 toto.py
```

Description : La commande `more` permet de visualiser le contenu d'un fichier texte page par page, facilitant la lecture de longs fichiers.

Syntaxe :

```
more [fichier]
```

Exemples :

```
invite/> more fichier.txt
```

Navigation :

- Space : Page suivante
- Enter : Ligne suivante
- q : Quitter

Commandes avancées sur les fichiers

- **cut** : Extrait des sections de chaque ligne d'un fichier
- **sort** : Trie les lignes d'un fichier
- **uniq** : Élimine les lignes dupliquées dans un fichier trié
- **wc** : Compte les lignes, mots et caractères d'un fichier
- **paste** : Concatène les lignes de même numéro de plusieurs fichiers
- **join** : Fusionne les lignes de deux fichiers sur un champ commun
- **sed** : Éditeur de flux pour transformer ou filtrer du texte
- **find** : Recherche des fichiers et exécute des actions sur eux
- **grep** : Recherche des motifs dans les fichiers

Exemple

Exemple d'un fichier que nous allons utiliser pour les commandes cut, sort, uniq et wc.

```
invite/> cat fichier.txt  
3;tutu;11  
1;toto;18  
1;toto;18  
4;titi;13  
2;tata;10
```

cut

Description : La commande `cut` permet d'extraire des sections spécifiques de chaque ligne d'un fichier.

Syntaxe :

```
cut [options] [fichier]
```

Exemples :

Afficher la première colonne :

```
invite/> cut -f1 -d";" fichier.txt
3
1
1
4
2
```

Afficher la première et troisième colonne :

```
invite/> cut -f1,3 -d";" fichier.txt
3;11
1;18
1;18
4;13
2;10
```

Description : Continuité de l'utilisation de la commande `cut` pour extraire des sections de texte.

Exemple : Afficher les trois premières colonnes :

```
invite/> cut -f1-3 -d";" fichier.txt  
3;tutu;11  
1;toto;18  
1;toto;18  
4;titi;13  
2;tata;10
```

Exemple : Afficher un caractère spécifique (par exemple, le premier caractère) :

```
invite/> cut -c1 fichier.txt  
3  
1  
1  
4  
2
```

Description : La commande `sort` trie les lignes d'un fichier selon différents critères.

Syntaxe :

```
sort [options] [fichier]
```

Exemples :

Trier par ordre numérique :

```
invite/> sort -n fichier.txt
1;toto;18
1;toto;18
2;tata;10
3;tutu;11
4;titi;13
```

Trier par la troisième colonne :

```
invite/> sort -k3 -t";" -n fichier.txt
2;tata;10
3;tutu;11
4;titi;13
1;toto;18
1;toto;18
```

Description : La commande `uniq` élimine les lignes dupliquées consécutives dans un fichier trié.

Syntaxe :

```
uniq [options] [fichier]
```

Exemples :

Éliminer les doublons :

```
invite/> uniq fichier.txt
3;tutu;11
1;toto;18
4;titi;13
2;tata;10
```

Compter les occurrences :

```
invite/> uniq -c fichier.txt
1 3;tutu;11
2 1;toto;18
1 4;titi;13
1 2;tata;10
```

Description : La commande `wc` (word count) compte les lignes, mots et caractères d'un fichier.

Syntaxe :

```
wc [options] [fichier]
```

Options Principales :

- `-l` : Compte les lignes.
- `-w` : Compte les mots.
- `-c` : Compte les caractères.

Exemples :

Compter les lignes :

```
invite/> wc -l fichier.txt  
5 fichier.txt
```

Compter les mots :

```
invite/> wc -w fichier.txt  
8 fichier.txt
```

Compter les caractères :

```
invite/> wc -c fichier.txt  
48 fichier.txt
```

Description : La commande **paste** concatène les lignes de même numéro de plusieurs fichiers en les séparant par un délimiteur (par défaut, une tabulation).

Exemples :

Fichiers Exemple :

janvier2012 :

```
Alimentation 50.00  
Eau 25.00  
Electricite 123.50  
Loyer 456.90  
Assurances 234.00
```

fevrier2012 :

```
Alimentation 67.00  
Eau 34.00  
Electricite 156.00  
Loyer 456.90  
Assurances 225.00
```


Description : Continuité de l'utilisation de la commande `paste` pour concaténer les lignes de deux fichiers.

Exemple :

```
invite/> paste janvier2012 fevrier2012
Alimentation 50.00 Alimentation 67.00
Eau 25.00 Eau 34.00
Electricite 123.50 Electricite 156.00
Loyer 456.90 Loyer 456.90
Assurances 234.00 Assurances 225.00
```

join

Description : La commande `join` fusionne les lignes de deux fichiers basées sur un champ commun.

Syntaxe :

```
join [options] fichier1 fichier2
```

Exemples :

Fichiers Exemple :

janvier2012 :

```
Alimentation 50.00  
Eau 25.00  
Electricite 123.50  
Loyer 456.90  
Assurances 234.00
```

fevrier2012 :

```
Alimentation 67.00  
Eau 34.00  
Electricite 156.00  
Loyer 456.90  
Assurances 225.00
```

Exemple :

```
invite/> join -1 1 -2 1 -t" " janvier2012 fevrier2012
Alimentation 50.00 67.00
Eau 25.00 34.00
Electricite 123.50 156.00
Loyer 456.90 456.90
Assurances 234.00 225.00
```

Explications :

- -1 1 : Champ de jointure du premier fichier (champ 1).
- -2 1 : Champ de jointure du deuxième fichier (champ 1).
- -t" " : Délimiteur utilisé (espace).

Description : La commande **sed** (Stream Editor) permet de modifier ou transformer le contenu d'un fichier texte selon des règles définies.

Syntaxe :

```
sed 's/[pattern]/[remplacement]/[options]' [fichier]
```

Exemples :

Remplacer un mot dans un fichier :

```
invite/> sed "s/Alimentation/ALIMENTATION/g" janvier2012
ALIMENTATION 50.00
Eau 25.00
Electricite 123.50
Loyer 456.90
Assurances 234.00
```

Options Utiles :

- **g** : Remplacer toutes les occurrences sur chaque ligne.
- **i** : Ignorer la casse.
- **p** : Afficher uniquement les lignes modifiées.

find

Description : La commande `find` permet de rechercher des fichiers et d'exécuter des actions sur eux.

Syntaxe de Base :

```
find [emplacement] [options] [critères]
```

Exemple : Chercher un fichier par son nom

```
invite/> find /usr/ -name "trash.svg"  
/usr/share/themes/adriend-light/cinnamon/trash.svg
```

Options Importantes :

- `-iname` : Recherche insensible à la casse.

Exemple avec `-iname` :

```
invite/> find /usr/ -iname "trash.svg"  
/usr/share/themes/adriend-light/cinnamon/trash.svg  
/usr/share/themes/adriend-light/cinnamon/TRASH.svg
```

Description : Continuité de l'utilisation de la commande `find` avec des critères avancés.

Exemples :

Chercher des fichiers de plus de 10Mo dans `/usr/bin` :

```
invite/> find /usr/bin -size +10M  
/usr/bin/fgfs /usr/bin/inkscape /usr/bin/clementine
```

Chercher des répertoires dont le nom contient "sm" dans `/var/log` :

```
invite/> find /var/log/ -type d -name "*sm*"
/var/log/samba/cores/smbd
```

Description : Continuation de l'utilisation avancée de `find`.

Exemple : Chercher des fichiers accédés il y a moins de 1 jour :

```
invite/> find /var/log/samba/ -atime -1  
/var/log/samba/ /var/log/samba/10.6.0.38.log /var/log/samba/10.5.26.125.log
```

Exemple : Chercher des fichiers accédés il y a plus de 90 jours :

```
invite/> find /var/log/samba/ -atime +90  
/var/log/samba/192.168.1.30.log /var/log/samba/calvin.log
```

Description : Utilisation avancée de `find` pour exécuter des commandes sur les fichiers trouvés.

Exemple : Copier tous les fichiers textes dans un autre répertoire

```
invite/> find /home/david/ -name "*.txt" -exec cp "{}" /home/david/toto/ \;
```

Explications :

- "{}" : Représente chaque fichier trouvé.
- \; : Termine la commande `-exec`.

Description : La commande **grep** recherche des motifs spécifiques dans les fichiers et affiche les lignes correspondantes.

Syntaxe :

```
grep [options] "motif" [fichier]
```

Exemples :

Rechercher un motif dans un fichier :

```
invite/> grep "chaine" fichier.txt
```

Rechercher un motif dans plusieurs fichiers :

```
invite/> grep "chaine" *.txt
```

```
fichier1.txt: Ceci est une chaine de test. fichier2.txt: Une autre chaine trouvée.
```

Options Utiles :

- **-i** : Recherche insensible à la casse.
- **-r** : Recherche récursive dans les répertoires.
- **-v** : Inverse la recherche, affiche les lignes qui ne contiennent pas le motif.

Section 3

Historique des commandes

Historique des commande - Partie 1

La commande **history** permet de voir rapidement les dernières commandes exécutées et ainsi pouvoir les re-exécutés.

- **Afficher l'historique**

```
invite/> history  
1 mkdir toto  
2 exit  
3 ls -la  
4 pwd
```

- **Afficher les n dernières lignes**

```
invite/> history 3  
2 exit  
3 ls -la  
4 pwd
```

Historique des commande - Partie 2

- Répéter la dernière commande

```
invite/> date
Dim 20 jan 2019 21:01:29 CET
invite/> !!
date
Dim 20 jan 2019 21:01:31 CET
```

- Répéter une commande spécifique

```
invite/> history 2
101 date
102 history 2
invite/> !101
date
Dim 20 jan 2019 21:01:35 CET
```

Historique des commande - Partie 3

- **Répéter une commande spécifique**

```
invite/> history
101 date
102 history 2
invite/> !-2
date
Dim 20 jan 2019 21:01:35 CET
```

- **Répéter des commandes précédentes**

En utilisant les touches haut et bas du clavier.

Historique des commande - Partie 4

- Répéter une commande qui commence par une chaîne de caractère

```
invite/> systemctl start httpd
invite/> systemctl stop chronyd
invite/> systemctl restart chronyd
invite/> !systemctl
restart chronyd
```

Attention : cela peut être dangereux de relancer la dernière commande si elle est différente de ce que vous attendiez.

```
invite/> !systemctl:p
systemctl restart chronyd
```

Historique des commande - Partie 5

- **Rechercher dans l'historique**

On peut rechercher une commande dans l'historique en utilisant les touches Control+R.

```
invite/> Appuyer sur les touches Control+R, cela permettra d'afficher le prompt  
reverse-i-search  
(reverse-i-search)`red': cat /etc/redhat-release
```

- **Substituer une commande de l'historique tout en gardant les paramètres**

```
invite/> ls anaconda-ks.cfg  
anaconda-ks.cfg  
invite/> vi !!:$
```

Historique des commande - Partie 6

- Substituer une commande de l'historique tout en gardant un paramètre

```
invite/> cp anaconda-ks.cfg anaconda-ks.cfg.bak  
invite/> vi !!:1  
vi anaconda-ks.cfg
```

```
invite/> cp anaconda-ks.cfg anaconda-ks.cfg.bak  
invite/> vi !!:2  
vi anaconda-ks.cfg.bak
```

- Supprimer l'historique

```
invite/> history -c
```


Section 4

Expressions régulières

- **Problématique** : Comment rechercher une chaîne de caractère
- Une expression régulière est une chaîne de caractère (motif, pattern) qui décrit selon une syntaxe précise un ensemble de chaîne de caractères précises.
- Avec les expressions régulières, vous pouvez :
 - rechercher des fichiers
 - rechercher une adresse email dans un fichier
 - vérifier si une adresse email est correcte

Exemple, pour rechercher toutes les adresse emails dans fichier.txt via la commande egrep :

```
invite/> egrep "[a-z.]*@[a-z-]*.[a-z]*" fichier.txt
```

Recherche d'un pattern

Recherche d'un pattern dans un texte :

Exemple	Matches	Not Match
petit	le petit chaperon rouge	le grand chaperon rouge

Exemple :

```
invite/> egrep "petit" fichier.txt
```

Quantificateur

Permet de compter le nombre de fois qu'une lettre peut apparaître

- $a?$: a peut apparaître 0 ou 1 fois
- a^+ : a peut apparaître au moins 1 fois
- a^* : a peut apparaître 0, 1 ou plusieurs fois
- $a\{n\}$: a doit apparaître n fois
- $a\{n,m\}$: a doit apparaître de n à m fois
- $a\{n,\}$: a doit apparaître au moins n fois
- $.$: n'importe quel caractère

Exemple	Matches	Not Match
bor?is	boris, bois	booris
o+h	oh, ooooh	oohhhhhh
peti*t	pett, petit, petiiiiit	petitttt
pet.t	petit, petat, petet	pett
9{3}	999	9, 99, 9999
9{1,3}	9999999	99999
9{2,}	99999999999	9

Exemple :

```
invite/> egrep "bor?is" fichier.txt
invite/> egrep "peti*t" fichier.txt
```

Dans les expressions régulières, le symbole `|` permet de spécifier une alternative : le motif correspond soit à l'expression à gauche du pipe, soit à celle à droite

- `|` : recherche quelque chose ou quelque chose

Exemple	Matches	Not Match
chien chat	chien, chat	oiseau

Exemple :

```
invite/> egrep "chien|chat" fichier.txt
```

Classe de caractère

Une classe de caractère permet de rechercher parmi plusieurs jeu de caractère, simplement en mettant les jeux de caractère entre crochet

- `[az]` : un seul caractère donné dans les brackets
- `[a-z]` : un caractère de la plage indiqué dans les brackets

Exemple	Matches	Not Match
<code>gr[ae]y</code>	gray, grey	grry
<code>gr[a-z]y</code>	gray, grey, ...	gr0y
<code>gr[0-9]y</code>	gr0y, gr9y, ...	gray
<code>gr[a-z0-9]y</code>	gray, grey, gr0y, gr9y	gr-y

Exemple :

```
invite/> egrep "gr[a-z]y" fichier.txt
```

Les symboles d'ancrage indiquent la position du motif dans la ligne

- `^` : début de ligne
- `$` : fin de ligne

Exemple :

```
invite/> egrep "^grey" fichier.txt
```